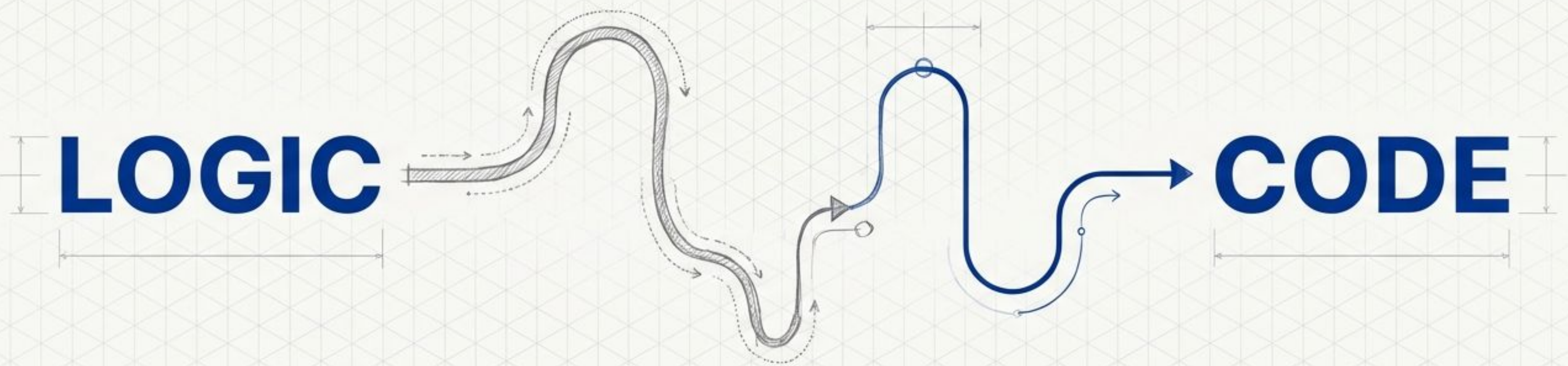
Flowcharts Before Coding

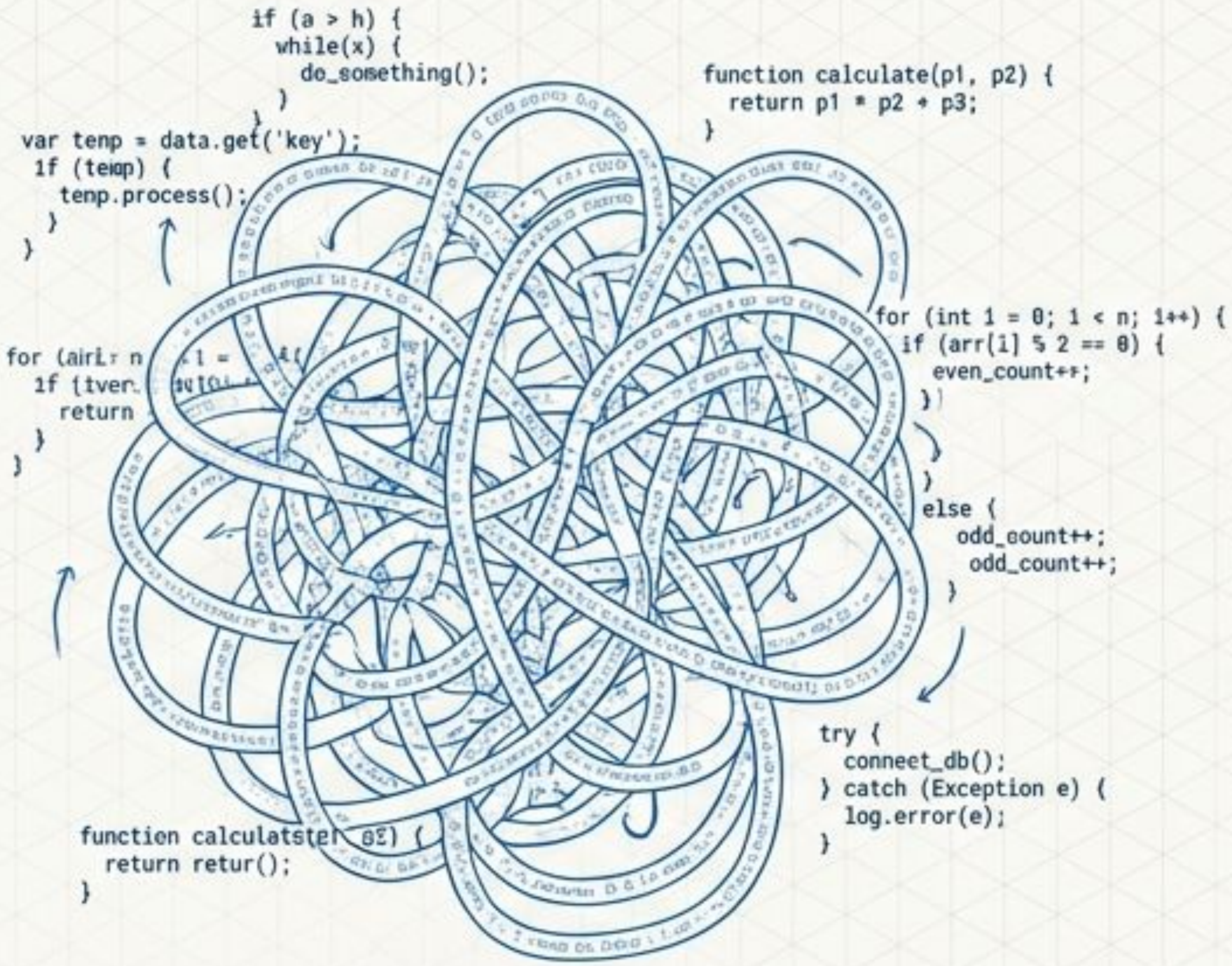
Visualizing Logic for Data Structures & Algorithms



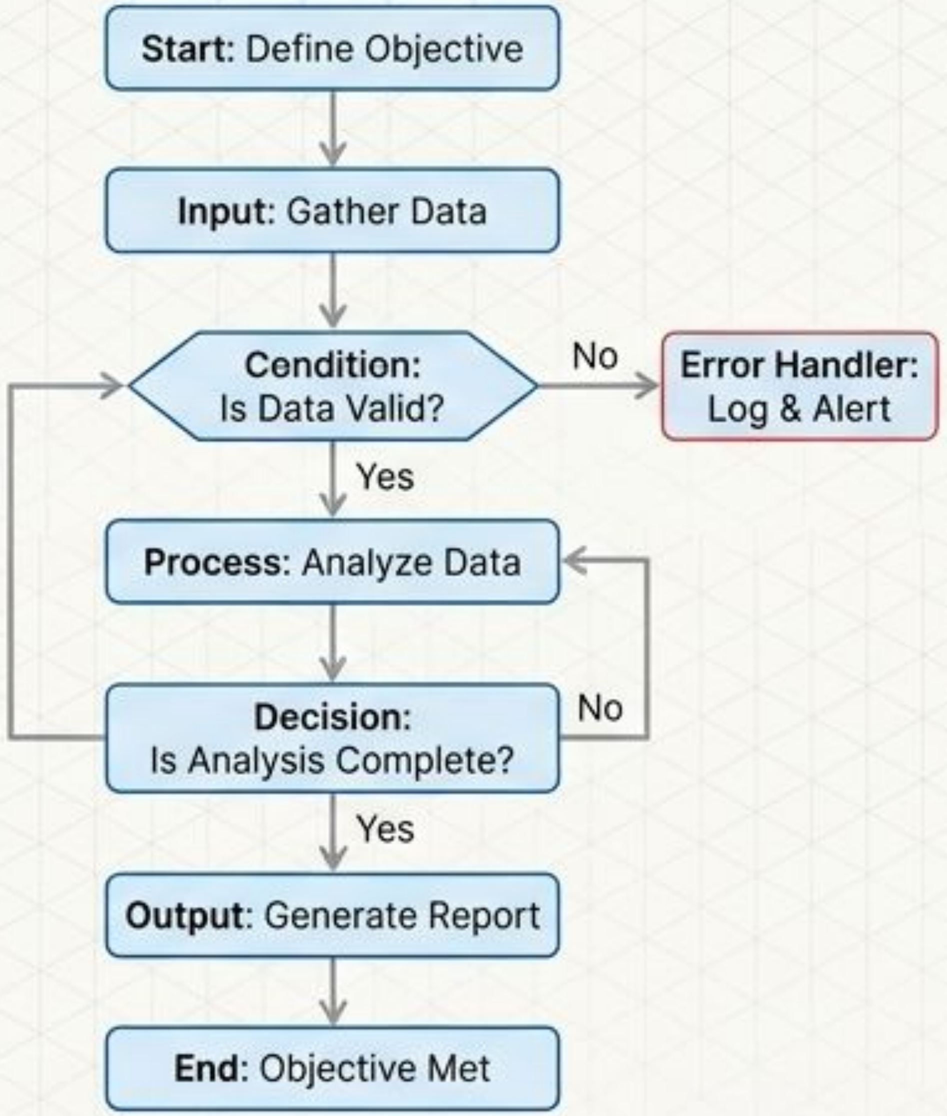
Course: Data Structures & Algorithms

Presenter: Adeel Atta

The Case for Visual Logic



The Problem: Spaghetti Code



The Solution: Structured Flow

Problem Solving First

Error Reduction

DSA Mastery

Visual Debugging

NotebookLM

The Toolkit: Basic Symbols

The alphabet of visual logic.

Start / End

Defines the boundaries of the program.

Input / Output

User interaction (Read/Print).

Process

Calculations, assignments, or actions.

Decision

A fork in the road (True/False).

Flow Line

The direction of execution.

software:
https://draw.io

The 7-Step Strategy

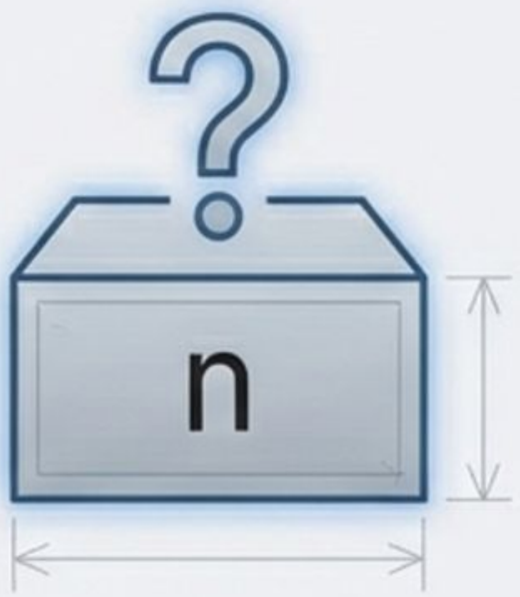
1. Understand Problem
2. Identify Inputs/ Outputs
3. Break into Steps
4. Draw Flowchart
5. Pseudocode
6. Write Code
7. Time Complexity

We are here: visualizing the blueprint before laying the bricks.

Example 1: Even or Odd

Problem Statement

Input a number. Check whether it is even or odd.

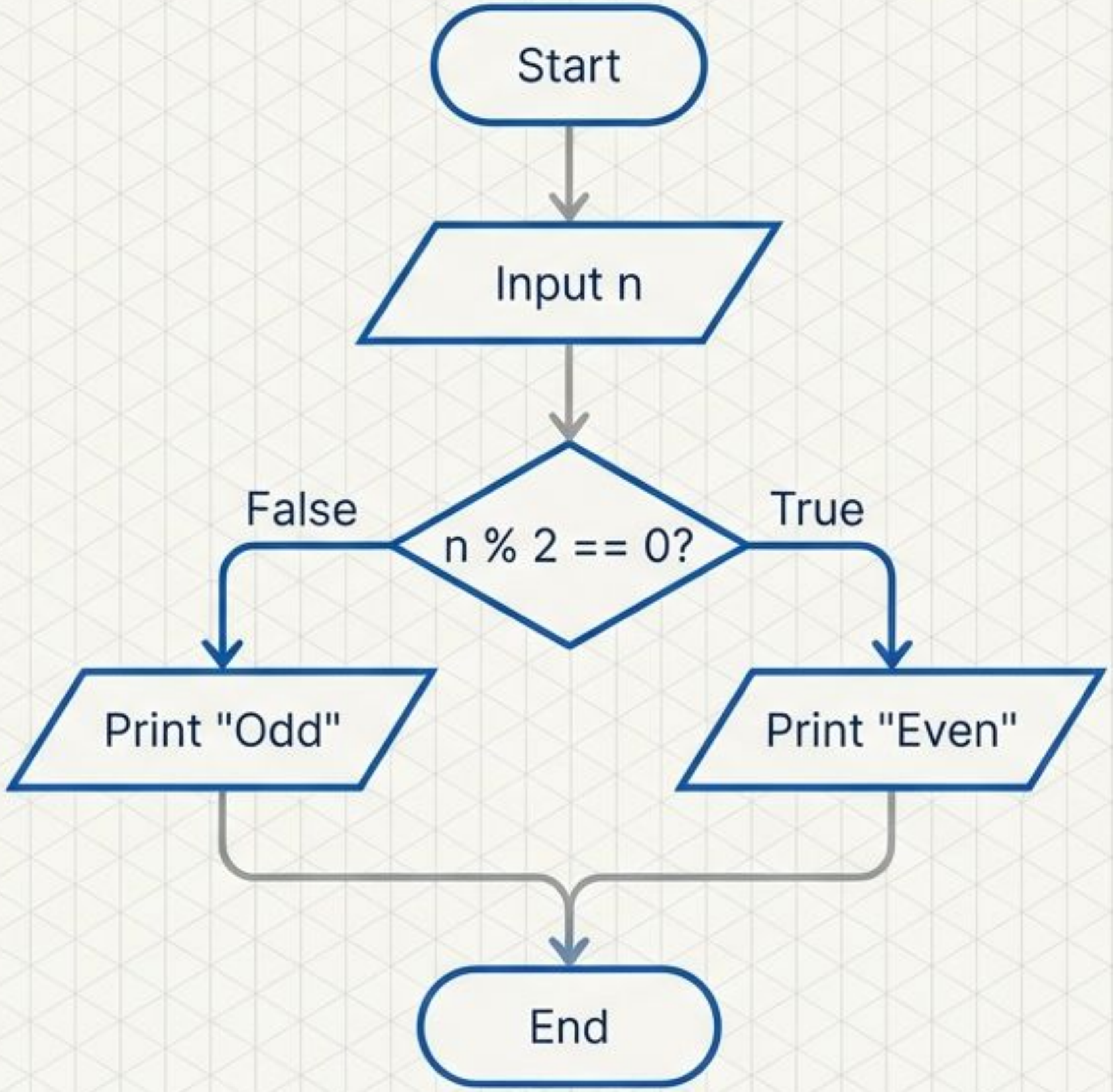


Logic Pattern: Branching (Even or Odd)

The Problem:
Input number n. Check if even or odd.

Logical Steps:

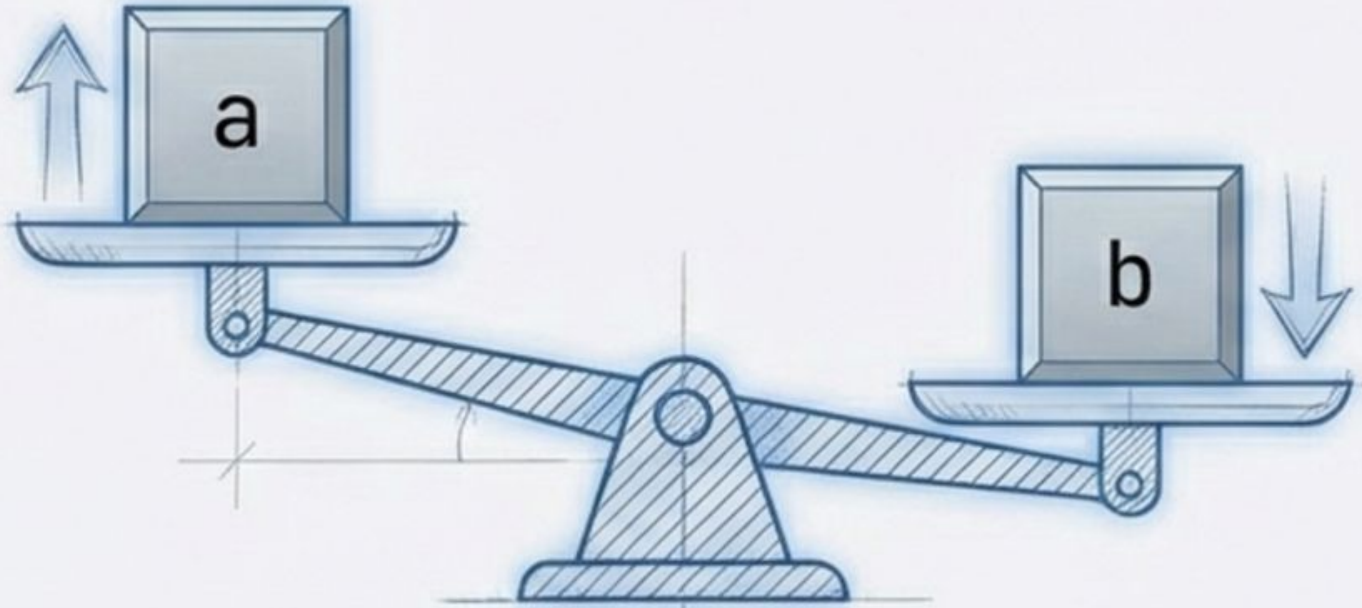
1. Start
2. Input n
3. Check: Is n % 2 == 0?
4. If True: Print 'Even'
5. If False: Print 'Odd'
6. End



Example 2: Find Maximum of Two Numbers

Problem Statement

```
Input two numbers a and b.
Print the larger number.
```



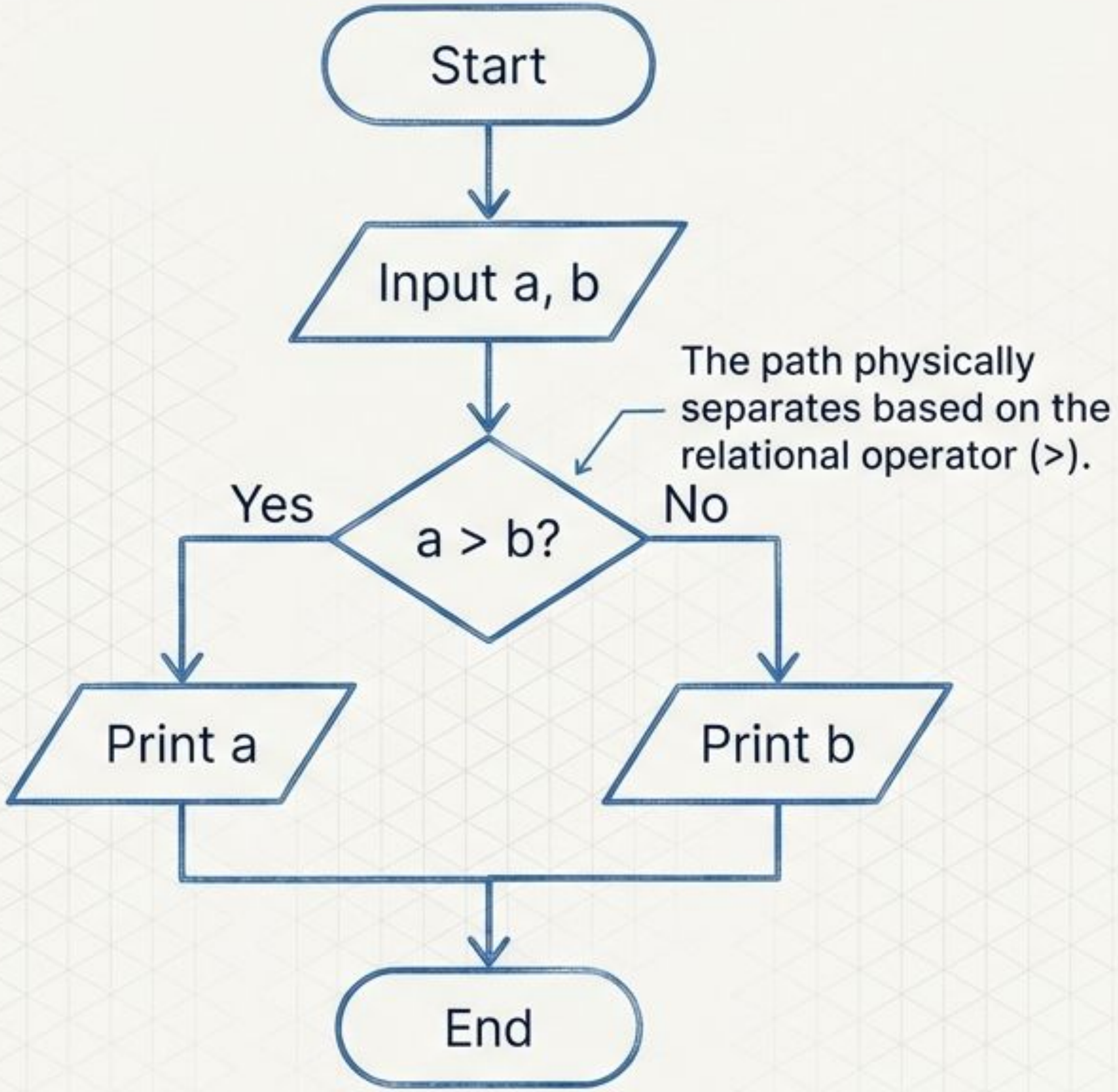
Logic Pattern: Comparison (Max of Two Numbers)

The Problem:

Input numbers a and b. Print the larger number.

Logical Steps:

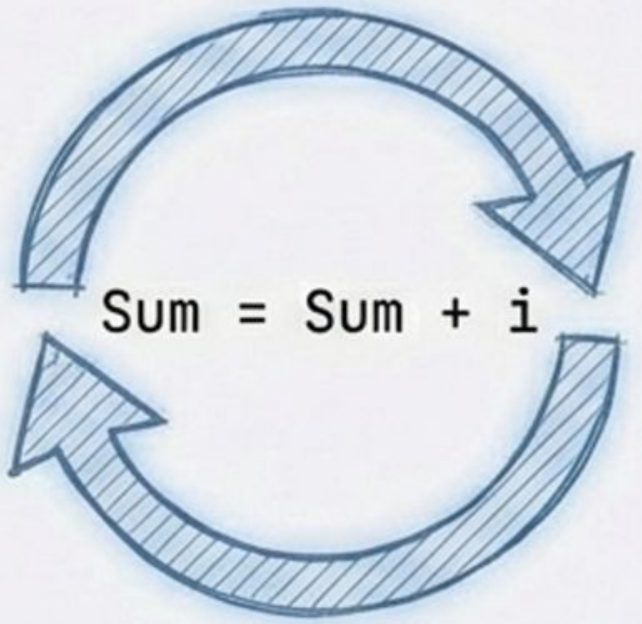
1. Start
2. Input a, b
3. Check: Is a > b?
4. If Yes: Print a
5. If No: Print b
6. End



Example 3: Sum of First N Numbers

Problem Statement (Loop)

Input n. Find sum of first
n natural numbers.



Logic Pattern: The Loop (Sum of First N)

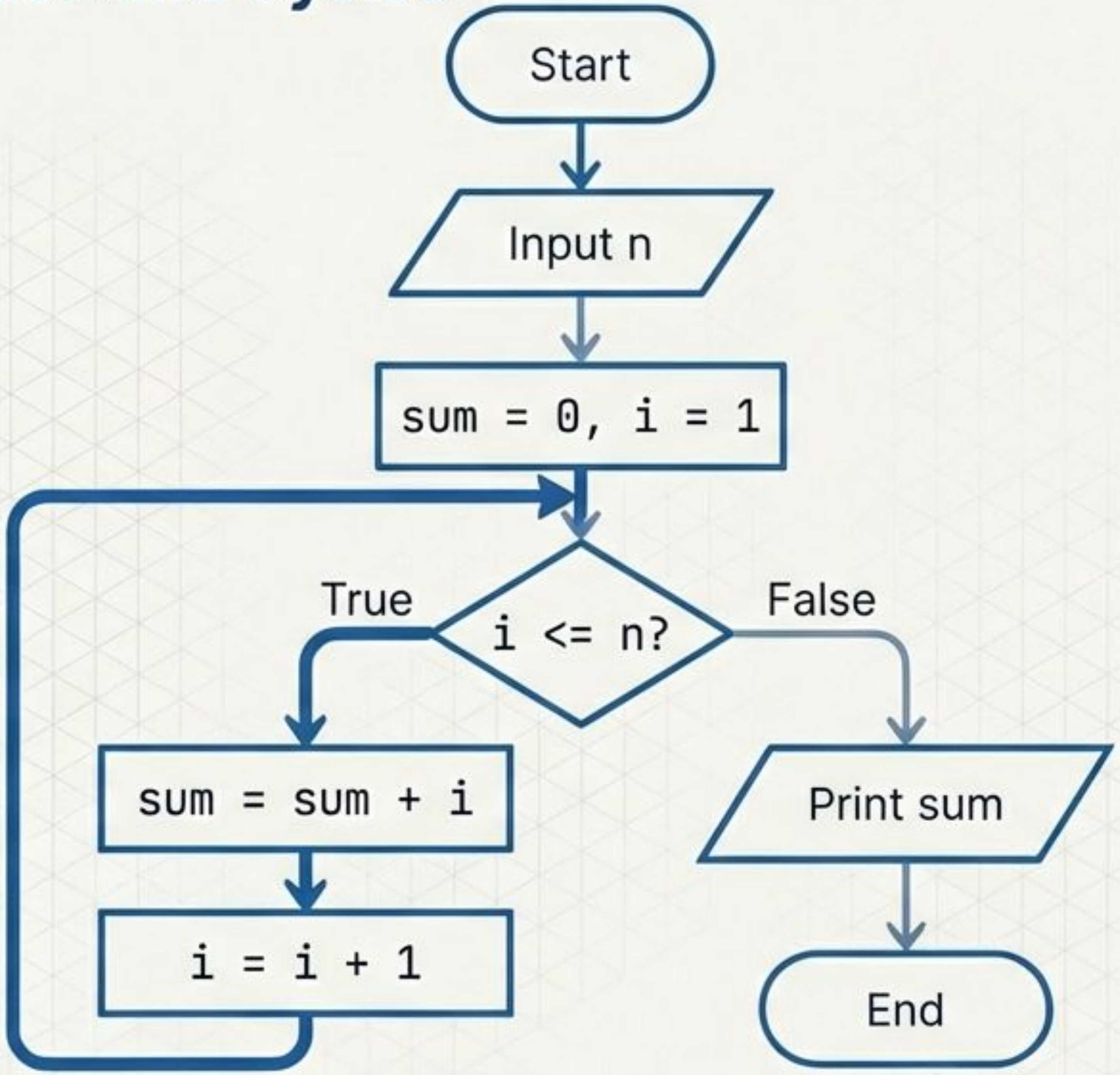
Visualizing Iteration and Cycles

The Problem:
Sum of first n natural numbers.

Variables:
sum = 0, i = 1

Loop Condition:
While i <= n

Update:
sum = sum + i, i = i + 1



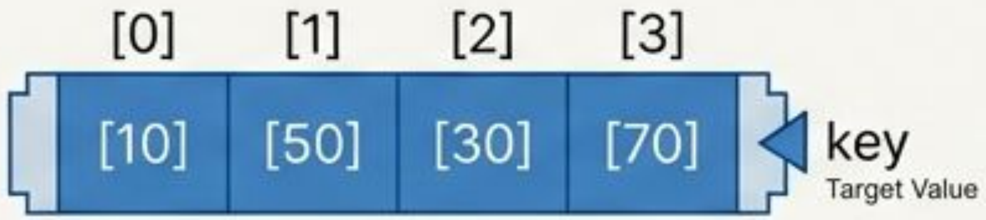
Example 4: Linear Search (DSA Example)

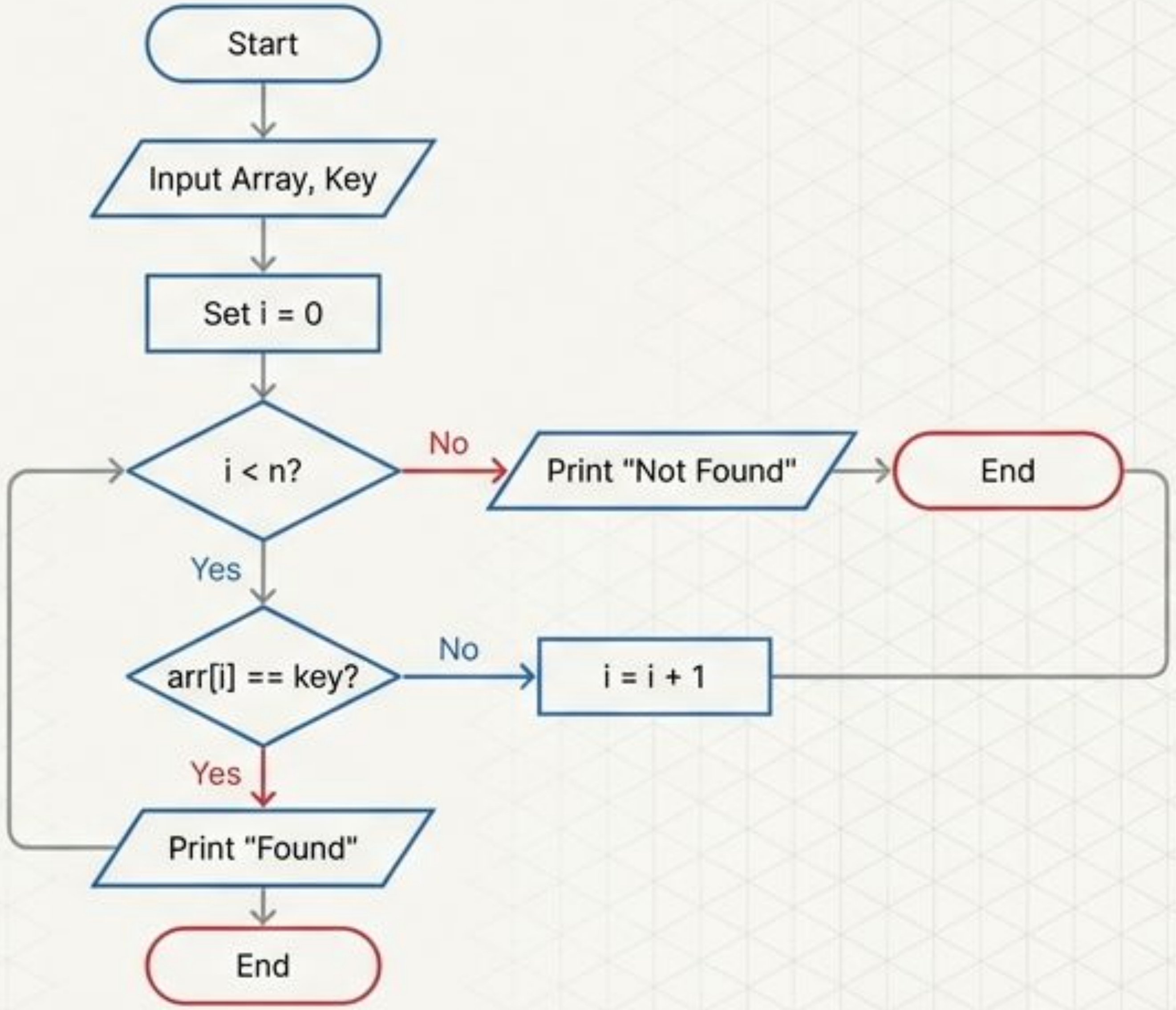
Problem Statement

Search for a key in an array using Linear Search.

Algorithm Visualization: Linear Search

Drafting Blue





NotebookLM

Skill Check: Workshop Tasks

Task 1: The Checker (Conditionals)

Create a flowchart that inputs a number and checks if it is:

- Positive (> 0)
- Negative (< 0)
- Zero (== 0)

Hint: This requires nested decision diamonds.

Task 2: The Builder (Loops)

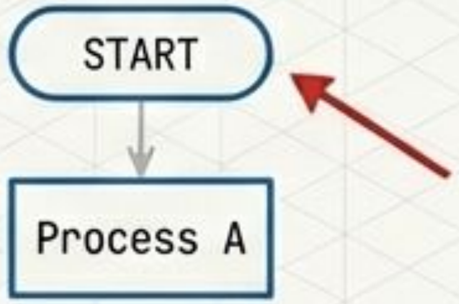
Create a flowchart to print the Multiplication Table of a given number.

- Input: 5
- Output: 5, 10, 15... 50

Hint: Requires a counter variable and a loop structure.

Debugging Logic: Common Mistakes

The Ghost



Don't

Always start/end with an Oval.

The Disconnect

Don't

Decisions must have outgoing paths.

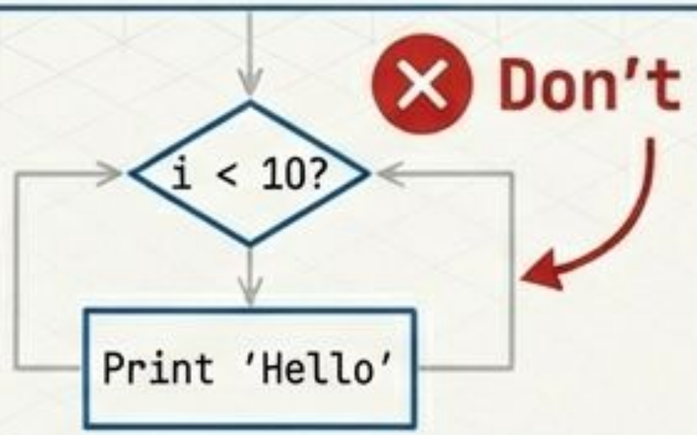
The Novelist

Don't

```
def calculate_average(data):
    total = sum(data)
    count = len(data)
    if count == 0: return 0
    return total / count
```

Use simple pseudocode, not syntax.

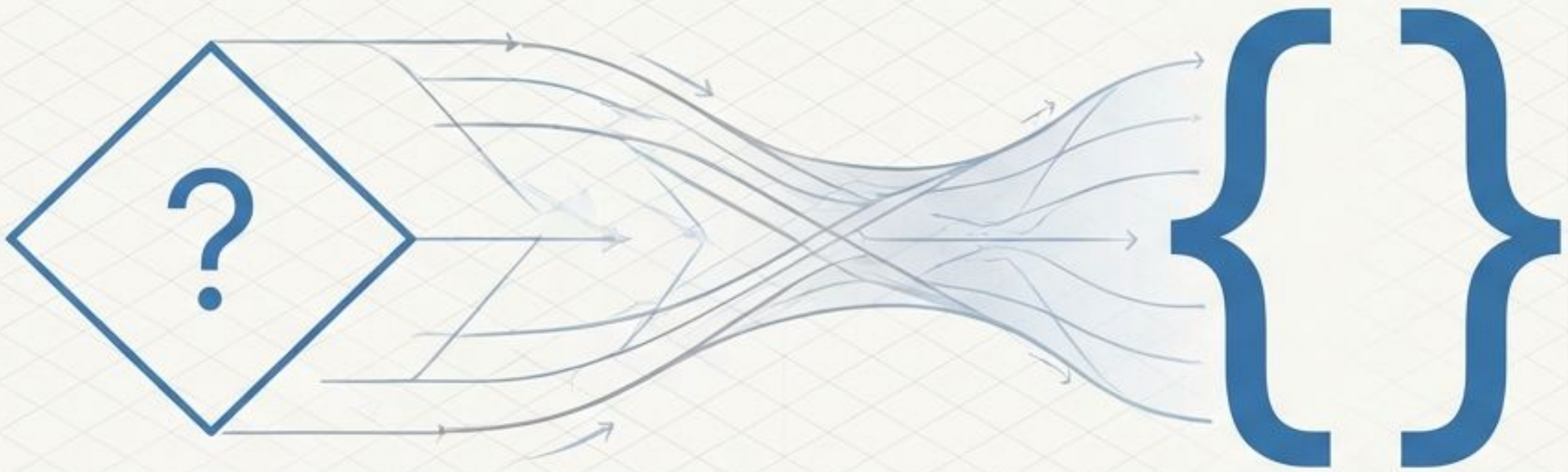
The Infinite Loop



Don't

Always update the loop variable.

Logic First, Syntax Second



Code is just the implementation of logic.
Flowcharts help you master the logic first.